A.F.P. Final

April 29th

The John Phillips Racket Association

https://github.com/jackp1377/MSF-Eval---Racket/tree/master/Project

What did we need?

Structs

- We need to be able to store and operate on state information

Gates

- The specific operations we apply to our structs to mutate them

Operations

- A broad category - includes various small helper functions, as well as essential functions for evaluating states

Circuit

- A larger syntax operation that organizes gates and an initial state into one runnable, functional syntax

https://github.com/jackp1377/MSF-Eval---Racket/tree/master/Project

Further Details...

STRUCTS

(struct state (vec neg-marker)))
(struct superposition (vec))

– State: contains qubit information for a single state
– Superposition: represents a superposition of states
– These structs are meant to be represented in a tree, with superpositions being nested into each other
  – In more detailed models, we introduce probability amplitudes to better model real situations

GATES

(CNOT <state> 0 1)
(Hadamard <state> 0)
– Various gates that operate on states/superpositions, recursing through the tree structure of a current state accordingly and adjusting qubits in state leaves
– Structs all contain their subtrees/leaves in vectors, making access and editing fairly quick
– Gate syntax definitions expand into full racket code that does this - this will be mentioned again later....

Further Details...

OPERATIONS

(flatten-superposition <superposition>)
(fill-negative <superposition>)
(vector-set <vector> <new item> <index>)

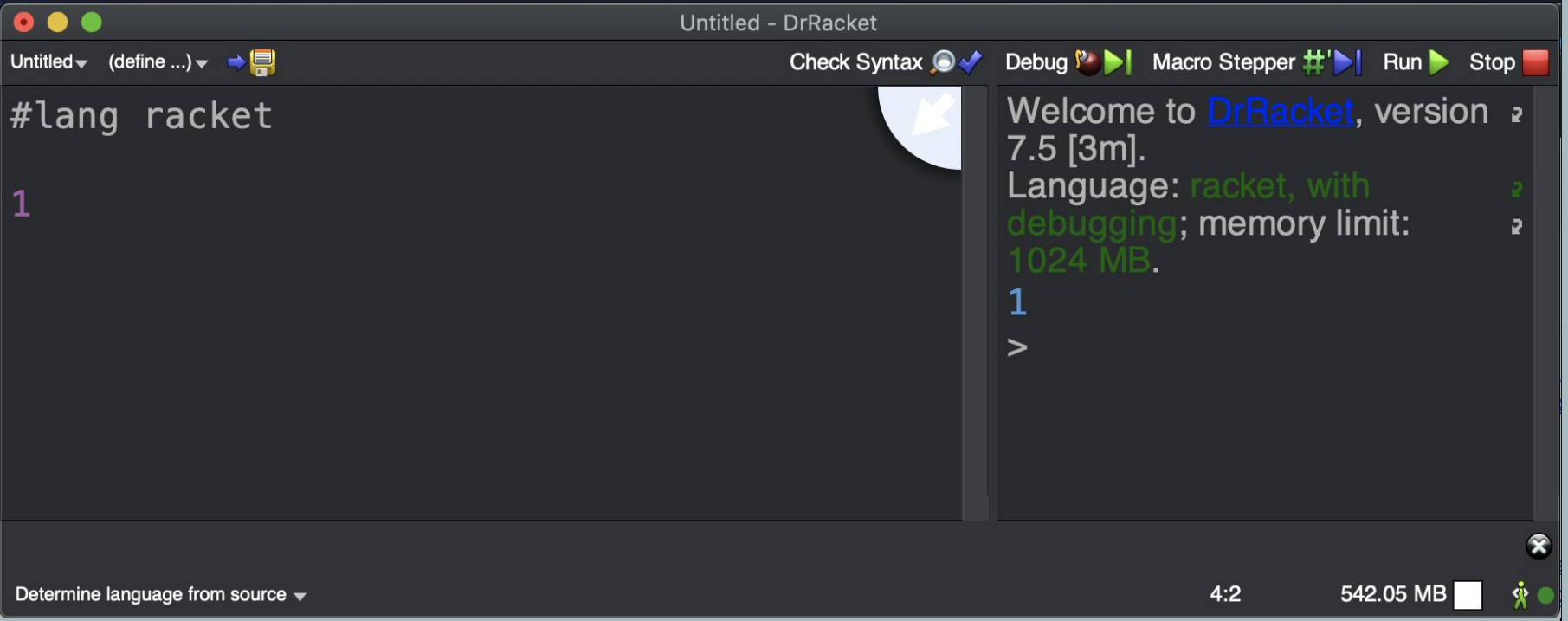
– Helpers: there's a bunch of miscellaneous helper functions in here, just to define some functionality to be used in the language
– Operations: there's a handful of operations that are pretty important for the simplification of states - these mutate our structs

CIRCUITS

– Makes a nested statement from the input

(circuit <input state> <gates> ...)
⇒
(<gate> (<gate> ... <index>) <index>)

– So, we can see that each gate in the list is nested to evaluate in order, with the input state being inserted as the base case into the middle



Code Examples...

Structs

- Defined twice so they can be referenced during both phases of compilation
- Structs get really weird in this regard … this led to some difficulty

```
(begin-for-syntax
    (struct state (vec neg-marker) #:transparent)
    (struct superposition (vec) #:transparent))

(struct state (vec neg-marker) #:transparent)
(struct superposition (vec) #:transparent)
```

Gates

- Recurses through state tree, operating on state leaves when found
- This particular gate is a simple not operation

```
(define-syntax X
    (lambda (stx)
        (syntax-parse stx
            ((_ in-state target)
                #`(letrec ((loop
                    (lambda (current targ frmr)
                        (match current
                            ((state state-vec _)
                                (loop (vector-ref state-vec targ) targ state-vec))
                            ((superposition sup-vec)
                                (superposition (for/vector ((v sup-vec)) (loop v targ sup-vec))))
                            ('W (state (vector-set frmr 'B targ) #f))
                            ('B (state (vector-set frmr 'W targ) #f))
                            ('w (state (vector-set frmr 'b targ) #f))
                            ('b (state (vector-set frmr 'w targ) #f))
                            ))))
                (loop in-state target '()))))))
```

Operations

- Mostly for utility in some form or another
- This example flattens a tree completely, making it much easier to compare and cancel states

```
(define-syntax flatten-superposition
    (lambda (stx)
        (syntax-parse stx
            ((_ in-super)
                (define vec-name (format-id stx "~a" (gensym 'vec)))
                #`(let ((#,vec-name (vector))) (letrec ((loop
                    (lambda (current)
                        (match current
                            ((state _ _)
                                (vector-add! #,vec-name current))
                            ((superposition sup-vec)
                                (begin (for ((s sup-vec)) (loop s))))
                            ((vector vecs syntax/ellipses)
                                (begin (for ((s vecs)) (loop s))))
                            (_
                                current)))))
                    (begin (loop in-super) (superposition #,vec-name)))))))
```

Circuits

- A syntax function that re-organizes gates and an input state into a single nested function
- Allows for a more intuitive representation of circuits

```
(define-syntax circuit
    (lambda (stx)
        (syntax-parse stx
            ((_ in-state gates ...)
                (letrec ((loop
                    (lambda (gate-list)
                        (cond
                            ((eq? (cdr gate-list) '())
                                (syntax-parse (car gate-list)
                                    ((gate num ...)
                                        ; (println #'gate)
                                        #'(gate in-state num ...))))
                            (else
                                (syntax-parse (car gate-list)
                                    ((gate num ...)
                                        #`(gate #,(loop (cdr gate-list)) num ...))))
                            ))))
                ; (println #'(list gates ...))
                (loop (reverse (syntax-e #'(gates ...)))))
                ))))
```

Some Notes...

- This code is based on **structs**
  - Done so we could quickly encapsulate states and such, as well as store them and easily sort through them in match statements
  - Ran into some difficulties in regards to matching across phases - this could probably be worked out somehow, but it wasn't something we prioritized in favor of getting the whole thing working

- Vectors are used inside states/superpositions to hold **subtrees**
  - Could have used lists, but vectors have a faster ref function and a lot of that happens in this code, so it seemed prudent to use them instead
  - Some of the vector helper functions have some overhead though (like converting to lists and back for append) so whatever time is saved by a quicker vector-ref may be counterbalanced by this

Wavefunction Collapse

In a superposition, equivalent states of opposite phase cancel out

This is the main appeal of quantum information, as it allows us interesting algorithms classical computing can only emulate (as we do)

Originally, collapse was handled with an expensive function that compared each state in a flattened tree to the entire superposition - this was genuinely too slow and barely ran

The solution was to integrate collapse into the hadamard gate, so that we don't need one of the layers of looping and can collapse states as the circuit is evaluated instead of at the end

Also, because qubits within states can be positive/negative interchangeably, it was easiest to represent phase with a boolean in the state struct

Address me

The Elephant in the Room

You may have noticed earlier that most of these operations expand into a bunch of racket code, as they're macros

The macro expander does NOT like this at all… running this program in racket is slow and the macro stepper can barely handle it

Were we to redo this project, it might be best to implement all of these gates as functions and forego making an entire language. That, or figure out a way to handle a lot of this logic without relying on expanding into a billion lines of racket code

After all, each gate function is an entire loop, and generating 50+ lines for loops that could have just been recycled is not preferable

Demo time!

Thank You!

The John Phillips Racket Association

https://github.com/jackp1377/MSF-Eval---Racket/tree/master/Project